Proyecto Final Etapa 4: Diseño e implementación de las API.

Autores: Génova, Candela – Massa, Luca – Kristensen, Franco

Docente: Fernandez Carbonell, Cesar Augusto

Facultad: Universidad Tecnológica Nacional (UTN) Extensión Aúlica Necochea.

Materia: Programación IV

28/10/2026

UNIVERSIDAD TECNOLÓGICA NACIONAL

4.1 Especificación de las API

Todas las rutas llevan el prefijo **/api**, trabajan con JSON (salvo la inscripción, que además recibe archivos) y devuelven los errores como un objeto con el campo error y un mensaje. La autenticación usa un token JWT (JSON Web Token), ya que si un usuario inicia sesión, simplemente el servidor emite un token y el cliente envía ese mismo token generado, de esa forma el servidor lo valida sin necesidad de consultar a la base de datos cada vez que inicia sesión, es enviado en el encabezado como: Authorization: Bearer <token>. Los niveles de acceso **son Público, Usuario, Personal y Admin**, y se indican entre paréntesis junto a cada ruta. Nunca se devuelve el campo contraseña.

Los parámetros de ruta se escriben :id y los de consulta ?campo=.

Autenticación y perfil(Creación del Usuario).

Verbo	Ruta	Acceso	Parámetros	Body esperado	Respuesta	Código de Estado
POST	**/api/auth/registro**	Público	-	{username, email, contraseña}	Usuario creado con rol **USUARIO**	201, 400, 409
POST	**/api/auth/login**	Público	-	{username, contraseña}	**{token, usuario:{id_usuario, nombre, rol}}**	200, 400, 401
GET	**/api/perfil**	Usuario	-	-	Datos del usuario logueado	200, 401
PUT	**/api/perfil**	Usuario		{username,email, contraseña}	Perfil actualizado	200, 400, 401
GET	**/api/usuarios**	personal, admin	consulta: rol, estado, q	-	lista de usuarios, sin contraseña	200, 400, 401, 403

PATCH	**/api/usuario/:id**	admin	ruta: id	{rol, estado}	usuario actualizado	200, 400, 401, 403, 404, 409(dejaria al sistema sin administradores)
DELETE	**/api/usuario/:id**	admin	ruta: id	-	usuario eliminado	204, 401, 403, 404, 409 (tiene alumno, personal o sugerencia)

Talleres

Verbo	Ruta	Acceso	Parámetros	Body esperado	Respuesta	Código de Estado
GET	**/api/talleres**	**Público**	**Consulta: barrio, de, edad**	**-**	Lista de talleres activos con barrio y cupo disponible	**200,400**
GET	**/api/talleres/:id**	**Público**	**Ruta: id**	**-**	**Un taller**	**200, 400, 404**
POST	**/api/talleres**	**Admin**	**-**	{nombre, descripcion, dia, hora_inicio, hora_fin, edad_minima, edad_maxima, id_barrio, cupo}	**Taller creado**	**201, 400, 401, 403**
PUT	**/api/talleres/:id**	**Admin**	**Ruta: id**	{nombre, descripcion, dia, hora_inicio, hora_fin, edad_minima,	**Taller actualizado**	**200, 400, 401, 403, 404**

				edad_maxima, id_barrio, cupo}		
DELETE	**/api/talleres/:id**	**Admin**	**Ruta: id**	**-**	**Taller eliminado**	**204, 400, 401, 403, 404, 409**

Barrios

Verbo	Ruta	Acceso	Parámetros	Body esperado	Respuesta	Código de Estado
GET	**/api/barrios**	**Público**	**-**	**-**	**Lista barrios**	**200**
POST	**/api/barrios**	**Admin**	**-**	**{nombre, descripcion}**	**Barrio creado**	**201, 400, 401, 403, 409**
PUT	**/api/barrios/:id**	**Admin**	**Ruta: id**	**{nombre, descripcion}**	**Barrio actualizado**	**200, 400, 401, 403, 404, 409**
DELETE	**/api/barrios/:id**	**Admin**	**Ruta: id**	**-**	**Barrio eliminado**	**204, 401, 403, 404, 409(tiene talleres o actividades asociados)**

Inscripciones

Las inscripciones las carga el personal a nombre del alumno: el alumno no se inscribe por sí mismo ni necesita una cuenta de usuario.

Verbo	Ruta	Acceso	Parámetros	Body esperado	Respuesta	Código de Estado
POST	**/api/inscripciones**	**personal, admin**	**-**	**id_alumno, id_taller, comentario**	{id_inscripcion, estado_inscripcion : PENDIENTE}	**201, 400, 401, 403, 404 (alumno o taller inexistente), 409 (ya inscripto o sin cupo)**
GET	**api/inscripciones/:id**	**personal , admin**	Consulta : estado, taller, alumno	**-**	**Inscripciones filtradas**	**200, 401, 403**
GET	**api/inscripciones/:id**	**personal, admin**	**Ruta: id**	**-**	**Detalle con datos del alumno, el taller, el personal que la cargó**	**200, 401, 403, 404**
PUT	**api/inscripciones**	**admin**	**Ruta: id**	**{estado_inscripcion: ACEPTADA o RECHAZADA, comentario}**	**inscripción actualizada**	**200, 400, 401, 403, 404, 409 (ya resuelta o sin cupo)**

Alumnos

Verbo	Ruta	Acceso	Parámetros	Body esperado	Respuestas	Código de Estado
GET	**/api/alumnos**	**Admin**	**dni, q**	**No requiere**	**Lista de alumnos**	**200, 401, 403**
GET	**/api/alumnos/:id**	**Admin**	**id, del alumno**	**No requiere**	**Alumno encontrado;** **inexistente**	**200;** **404**
POST	**/api/alumnos**	**Admin**	**ninguno**	**{nombre, apellido, dni, telefono,tiene_beca, estudios_completos, colegio}**	**Alumno creado;** **datos inválidos;** **registro duplicado**	**201;** **400;** **409**
PUT	**/api/alumnos/:id**	**Admin**	**id, del alumno**	**{nombre, apellido, dni, telefono,tiene_beca, estudios_completos, colegio}**	**Actualizado;** **no existente;** **sin permisos**	**200;** **404;** **403**
DELETE	**/api/alumnos/:id**	**Admin**	**id, del alumno**	**No requiere**	**Eliminado;** **no existe**	**200;** **404.** **400.**

Personal

Verbo	Ruta	Acceso	Parámetros	Body esperado	Respuesta	Código de Estado
GET	**/api/personal**	**Admin**	**Ninguno**	**No requiere**	**Lista de personal**	**200**
GET	**/api/personal/:id**	**Admin**	**id, del integrante**	**No requiere**	**Integrante encontrado; no existe**	**200; 404**
POST	**/api/personal**	**Admin**	**ninguno**	**{id_usuario,nombre, apellido, dni, telefono, funcion}**	**Integrante creado; datos invalidos**	**201; 400**
PUT	**/api/personal/:id**	**Admin**	**id, del integrante**	**{id_usuario, nombre, apellido, dni, telefono,funcion**}	**Actualizado; no existente**	**200; 404**
DELETE	**/api/personal/:id**	**Admin**	**id, del integrante**	**No requiere**	**Eliminado; no existe**	**204; 404.**

Asistencias

Verbo	Ruta	Acceso	Parámetros	Body esperado	Respuesta	Código de Estado

GET	**/api/asistencias**	**Admin**	**id_alumno, id_taller, fecha**	**No requiere**	**Lista de asistencias;**	**200**
GET	**/api/asistencias/:id**	**Admin**	**id, de la asistencia**	**No requiere**	**Asistencia encontrada; no existente**	**200; 404**
POST	**/api/asistencias**	**Admin**	**ninguno**	**{ id_alumno,id_taller, fecha, presente}**	**Asistencia registrada; datos inválidos**	**201;** **400**
PUT	**/api/asistencias/:id**	**Admin**	**id, de la asistencia**	**{fecha, presente}**	**Actualizado; no existente**	**200;** **404**
DELETE	**/api/asistencias/:id**	**Admin**	**id, de la asistencia**	**No requiere**	**Eliminado; no existe**	**204;** **404.**

Seguimientos

Verbo	Ruta	Acceso	Parámetros	Body esperado	Respuesta	Código de Estado
GET	**/api/seguimientos**	**Admin**	**id_alumno**	**No requiere**	**Lista de seguimiento; sin permisos**	**200; 403**
GET	**/api/seguimientos/:id**	**Admin**	**id, del seguimiento**	**No requiere**	**Seguimiento encontrado; no existe**	**200; 404**

POST	**/api/seguimientos**	**Admin**	**ninguno**	**{ id_alumno, id_personal, fecha, observacion}**	**Seguimiento creado; datos inválidos**	**201; 400**
PUT	**/api/seguimientos/:id**	**Admin**	**id, del seguimiento**	**{fecha, descripción}**	**Actualizado; no existente**	**200; 404**
DELETE	**/api/seguimientos/:id**	**Admin**	**id, del seguimiento**	**No requiere**	**Eliminado; no existe**	**204; 404.**

Actividades

Verbo	Ruta	Acceso	Parámetros	Body esperado	Respuesta	Código de Estado
GET	**/api/actividades**	**Público**	**No requiere**	**No requiere**	**Lista de noticias o publicaciones**	**200, listado; 400, filtros inválidos**
GET	**/api/actividades/:id**	**Público**	**id, de la actividad**	**No requiere**	**encontrada, inexistente**	**200; 400**
POST	**/api/actividades**	**Admin**	**ninguno**	**{id_barrio,**	**Actividad creada;**	**201;**

				id_taller, nombre, descripción, fecha, estado}	**datos inválidos**	**400**
PUT	**/api/actividades/:id**	**Admin**	**id, de la actividad**	**{nombre, descripción, fecha, estado**}	**Actualizada; no existente**	**200;** **404**
DELETE	**/api/actividades/:id**	**Admin**	**id de la actividad**	**No requiere**	**eliminada; no existe**	**204;** **404.**

Noticias.

Verbo	Ruta	Acceso	Parámetros	Body esperado	Respuesta	Código de Estado
GET	**/api/noticias**	**Público**	**Opcionales: buscar, fecha**	**No requiere**	**Lista de noticias o publicaciones**	**200, listado; 400, filtros inválidos**
GET	**/api/noticias/:id**	**Público**	**id, de la noticia**	**No requiere**	**encontrada, inexistente**	**200;** **400**

POST	**/api/noticias**	**Admin**	**ninguno**	**{titulo, contenido, fecha_publicacion, imagen, estado}**	**Noticias creado**	**201 creada; 400 inválidos; 401 sin sesión; 403 sin permiso**
PUT	**/api/noticias/:id**	**Admin**	**id de la noticia**	**{titulo, contenido, fecha, imagen, estado}**	**actualizada, inválidos, sin sesión, sin permisos, inexistente**	**200; 400; 401; 403; 404**
DELETE	**/api/noticias/:id**	**Admin**	**id de la noticia**	**No requiere**	**eliminada, sin sesión, sin permiso, inexistente**	**204; 401; 403; 404.**

Sedes.

Verbo	Ruta	Acceso	Parámetros	Body esperado	Respuesta	Código de Estado
GET	**/api/sedes**	**Público**	**No requiere**	**No requiere**	**Listado**	**200**
GET	**/api/sedes/:id**	**Público**	**id, de la sede**	**No requiere**	**encontrada, inexistente**	**200; 400**

POST	**/api/sedes**	**Admin**	**ninguno**	**{nombre, direccion ,descripcion}**	**Noticias creado**	**201 creada; 400 inválidos; 401 sin sesión; 403 sin permiso**
PUT	**/api/sedes /:id**	**Admin**	**id de la sede**	**{nombre, direccion ,descripcion}**	**actualizada, no existente**	**200; 400;**
DELETE	**/api/sedes /:id**	**Admin**	**no requiere**	**no requiere**	**eliminada, no existe**	**204; 404.**

Redes Sociales.

Verbo	Ruta	Acceso	Parametros	Body Esperado	Respuesta	Codigo de estado
GET	**/api/sedes /:idSede/redes**	**publico**	**ruta: idSede**	**-**	**redes sociales activas de la sede**	**200, 404**
POST	**/api/sedes /:idSede/redes**	**admin**	**ruta: idSede**	**{id_sede, tipo, url, estado}**	**red social creada en esa sede**	**201, 400, 401, 403, 404**
PUT	**/api/redes /:id**	**admin**	**ruta: id**	**{tipo, url, estado}**	**red social actualizada**	**200, 400, 401, 403, 404**
DELETE	**/api/redes /:id**	**admin**	**ruta: id**	**-**	**red social eliminada**	**204, 401, 403, 404**

Sugerencias

Verbo	Ruta	Acceso	Parámetros	Body esperado	Respuesta	Código de Estado
GET	**/api/sugerencias**	**Admin**	**Ninguno**	**No requiere**	**Listado de sugerencias; sin permisos**	**200; 404**
GET	**/api/sugerencias/:id**	**Admin**	**id, de la sugerencia**	**No requiere**	**sugerencia encontrada, no inexistente**	**200; 404**
POST	**/api/sugerencias**	**Admin/ Usuario**	**ninguno**	**{id_usuario, mensaje,fecha, estado}**	**Sugerencia enviada; datos inválidos**	**201; 400**
PUT	**/api/sugerencias/:id**	**Admin**	**id de la sugerencia**	**{mensaje ,estado}**	**actualizada; datos invalido; no existente**	**200; 400; 404**
DELETE	**/api/sugerencias/:id**	**Admin**	**id de la sugerencia**	**no requiere**	**eliminada, no existe**	**204; 404.**

4.2 CRUD completo de las entidades del sistema

En esta sección se verifica el funcionamiento de las operaciones CRUD de las entidades del sistema mediante pruebas realizadas con un cliente REST (Postman). Se registran los códigos de estado HTTP esperados, los obtenidos durante las pruebas y el resultado de cada operación, con el objetivo de comprobar que la API responde correctamente tanto ante solicitudes exitosas como ante errores.

Entidad: Alumnos

Operación	Método	Endpoint	Códigos esperados	Código obtenido	Resultado

Listar	GET	api/alumnos	200, 401, 403	200	Correcto
Obtener por ID	GET	api/alumnos/:id	200, 404	200 y 404	Correcto
Crear	POST	api/alumnos	201, 400, 403	201 y 400	Correcto
Actualizar	PUT	api/alumnos/:id	200, 404, 403	200	Correcto
Eliminar	DELETE	api/alumnos/:id	200, 404, 400	200	Correcto

Entidad: Usuarios

Operación	Método	Endpoint	Códigos esperados	Código obtenido	Resultado
Listar	GET	api/usuarios	200, 401, 403	200	Correcto
Obtener por ID	GET	api/usuarios/:id	200, 404	200 y 404	Correcto
Crear	POST	api/usuarios	201, 400, 403	201 y 400	Correcto
Actualizar	PUT	api/usuarios/:id	200, 404, 403	200	Correcto
Eliminar	DELETE	api/usuarios/:id	200, 404, 400	200	Correcto

Entidad: Personal

Operación	Método	Endpoint	Códigos esperados	Código obtenido	Resultado
Listar	GET	api/personal	200, 401, 403	200	Correcto
Obtener por ID	GET	api/personals/:id	200, 404	200 y 404	Correcto
Crear	POST	api/personal	201, 400,	201 y 400	Correcto

			403		
Actualizar	PUT	api/personals/:id	200, 404, 403	200	Correcto
Eliminar	DELETE	api/personal/:id	200, 404, 400	200	Correcto

Entidad: Barrios

Operación	**Método**	**Endpoint**	**Códigos esperados**	**Código obtenido**	**Resultado**
Listar	GET	api/barrios	200, 401, 403	200	Correcto
Obtener por ID	GET	api/barrios/:id	200, 404	200 y 404	Correcto
Crear	POST	api/barrios	201, 400, 403	201 y 400	Correcto
Actualizar	PUT	api/barrios/:id	200, 404, 403	200	Correcto
Eliminar	DELETE	api/barrios/:id	200, 404, 400	200	Correcto

Entidad: Talleres

Operación	**Método**	**Endpoint**	**Códigos esperados**	**Código obtenido**	**Resultado**
Listar	GET	api/talleres	200, 401, 403	200	Correcto
Obtener por ID	GET	api/talleres/:id	200, 404	200 y 404	Correcto
Crear	POST	api/talleres	201, 400, 403	201 y 400	Correcto
Actualizar	PUT	api/talleres/:id	200, 404, 403	200	Correcto
Eliminar	DELETE	api/talleres/:id	200, 404, 400	200	Correcto

Entidad: Inscripciones

Operación	Método	Endpoint	Códigos esperados	Código obtenido	Resultado
Listar	GET	api/inscripciones	200, 401, 403	200	Correcto
Obtener por ID	GET	api/inscripciones/:id	200, 404	200 y 404	Correcto
Crear	POST	api/inscripciones	201, 400, 403	201 y 400	Correcto
Actualizar	PUT	api/inscripciones/:id	200, 404, 403	200	Correcto
Eliminar	DELETE	api/inscripciones/:id	200, 404, 400	200	Correcto

Entidad: Asistencias

Operación	Método	Endpoint	Códigos esperados	Código obtenido	Resultado
Listar	GET	api/asistencias	200, 401, 403	200	Correcto
Obtener por ID	GET	api/asistencias/:id	200, 404	200 y 404	Correcto
Crear	POST	api/asistencias	201, 400, 403	201 y 400	Correcto
Actualizar	PUT	api/asistencias/:id	200, 404, 403	200	Correcto
Eliminar	DELETE	api/asistencias/:id	200, 404, 400	200	Correcto

Entidad: Seguimientos

Operación	Método	Endpoint	Códigos esperados	Código obtenido	Resultado
Listar	GET	api/seguimie	200, 401,	200	Correcto

		ntos	403		
Obtener por ID	GET	api/seguimientos/:id	200, 404	200 y 404	Correcto
Crear	POST	api/seguimientos	201, 400, 403	201 y 400	Correcto
Actualizar	PUT	api/seguimientos/:id	200, 404, 403	200	Correcto
Eliminar	DELETE	api/seguimientos/:id	200, 404, 400	200	Correcto

Entidad: Sedes

Operación	**Método**	**Endpoint**	**Códigos esperados**	**Código obtenido**	**Resultado**
Listar	GET	api/sedes	200, 401, 403	200	Correcto
Obtener por ID	GET	api/sedes/:id	200, 404	200 y 404	Correcto
Crear	POST	api/sedes	201, 400, 403	201 y 400	Correcto
Actualizar	PUT	api/sedes/:id	200, 404, 403	200	Correcto
Eliminar	DELETE	api/sedes/:id	200, 404, 400	200	Correcto

Entidad: Redes

Operación	**Método**	**Endpoint**	**Códigos esperados**	**Código obtenido**	**Resultado**
Listar	GET	api/redes	200, 401, 403	200	Correcto
Obtener por ID	GET	api/redes/:id	200, 404	200 y 404	Correcto
Crear	POST	api/redes	201, 400, 403	201 y 400	Correcto
Actualizar	PUT	api/redes/:id	200, 404, 403	200	Correcto

Eliminar	DELETE	api/redes/:id	200, 404, 400	200	Correcto

Entidad: Noticias

Operación	**Método**	**Endpoint**	**Códigos esperados**	**Código obtenido**	**Resultado**
Listar	GET	api/noticias	200, 401, 403	200	Correcto
Obtener por ID	GET	api/noticias/:id	200, 404	200 y 404	Correcto
Crear	POST	api/noticias	201, 400, 403	201 y 400	Correcto
Actualizar	PUT	api/noticias/:id	200, 404, 403	200	Correcto
Eliminar	DELETE	api/noticias/:id	200, 404, 400	200	Correcto

Entidad: Sugerencias

Operación	**Método**	**Endpoint**	**Códigos esperados**	**Código obtenido**	**Resultado**
Listar	GET	api/sugerencias	200, 401, 403	200	Correcto
Obtener por ID	GET	api/sugerencias/:id	200, 404	200 y 404	Correcto
Crear	POST	api/sugerencias	201, 400, 403	201 y 400	Correcto
Actualizar	PUT	api/sugerencias/:id	200, 404, 403	200	Correcto
Eliminar	DELETE	api/sugerencias/:id	200, 404, 400	200	Correcto

Entidad: Actividades

Operación	**Método**	**Endpoint**	**Códigos esperados**	**Código obtenido**	**Resultado**
Listar	GET	api/actividades	200, 401, 403	200	Correcto
Obtener por ID	GET	api/actividades/:id	200, 404	200 y 404	Correcto
Crear	POST	api/actividades	201, 400, 403	201 y 400	Correcto
Actualizar	PUT	api/actividades/:id	200, 404, 403	200	Correcto
Eliminar	DELETE	api/actividades/:id	200, 404, 400	200	Correcto

4.4 Búsqueda: la operación que da servicio al buscador de R4.

El buscador global es una funcionalidad de consulta de la API y su objetivo es permitir la búsqueda de información mediante el parámetro obligatorio "**q"**, devolviendo los resultados agrupados por tipo de contenido: talleres, noticias, actividades y barrios.

Para su implementación se dispone de una ruta(routes), un controlador(controllers) y un repositorio específico(data), el repositorio consulta las tablas en la base de datos, sin necesidad de crear una tabla adicional para el buscador.

Verbo	Ruta	Acceso	Parámetros	Body esperado	Respuesta	Código de Estado
GET	**/api/buscador?q=(ejemplo: pintura)**	**Público**	**consulta: q(obligatorio)**	**No requiere**	**{talleres, noticias, actividades, barrios} con los resultados de cada tipo**	**200, 400(q vacio, no se encontró resultados)**

4.5 Instrucciones de ejecución en el README.md, verificadas en una máquina distinta de aquella en la que se desarrolló.

***Instalar las dependencias**

1. Abrir la terminal en VSCODE y posicionarse en la raiz /codigo y luego /api.
2. Ejecutar el comando: npm install

Este comando instala las dependencias declaradas en package.json y genera la carpeta node_modules, donde se almacenan los módulos necesarios para ejecutar la aplicación.

Los principales modulos y mas importantes que utilizamos son:
express, pg, dotenv, bcryptjs, jsonwebtoken

***Configurar las variables de entorno**

Una vez configurado los modulos, se requiere crear un archivo nuevo llamado .env en la misma raiz, copiando y pegando los valores que estan en .env.ejemplo y adaptandolos a la configuración del equipo en el que se encuentra(La base de datos).

***Iniciar el servidor**

Una vez instaladas las dependencias y configurada la base de datos, iniciar la API con el comando correspondiente definido en package.json, en este caso: npm run dev.

***Creación del usuario administrador**

El proyecto cuenta con un comando para crear un usuario con rol de administrador:
npm run admin:crear

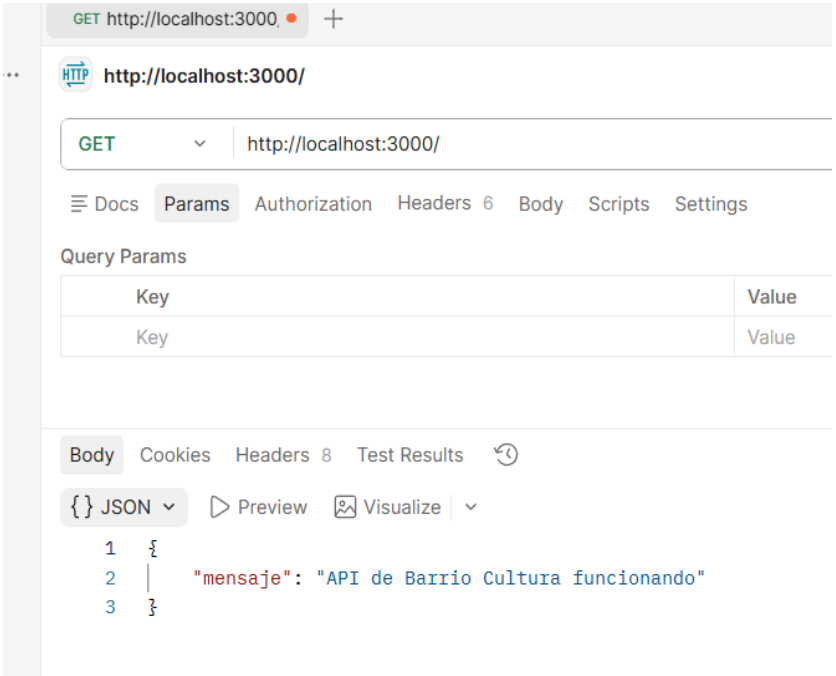
Básicamente lo que hace es generar un usuario utilizando las credenciales del .env y el crear-admin.js que se encarga de hacer el INSERT de un usuario nuevo con rol ADMINISTRADOR por defecto, permite usar los endpoints principales de la API.

Por motivos de seguridad, los usuarios registrados normalmente no pueden modificar su propio rol para obtener permisos de administrador. La asignación de este rol se realiza mediante el procedimiento específico de creación del administrador.

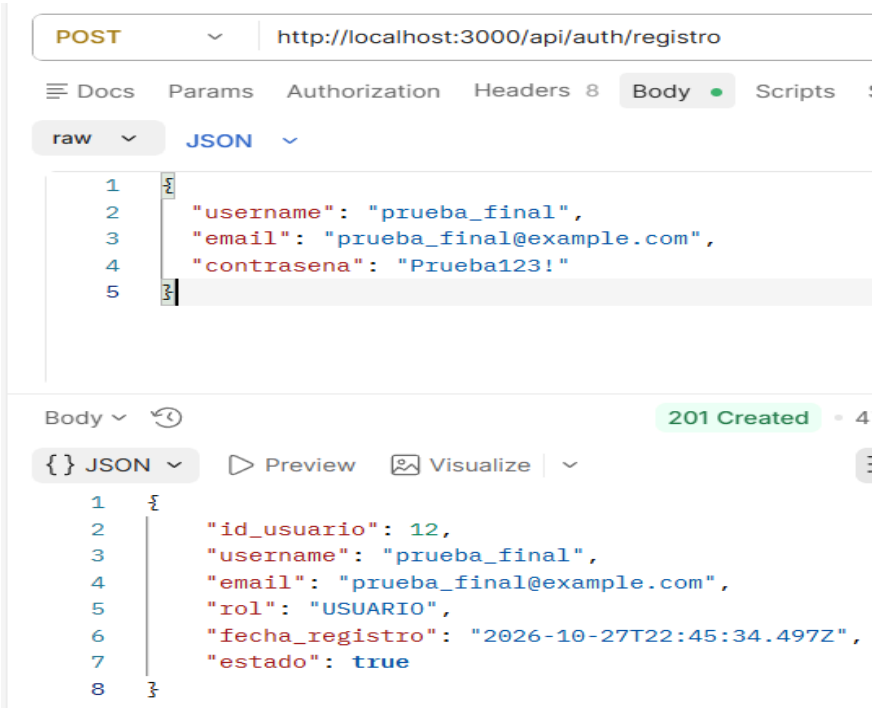
4.6 Colección de pruebas de los endpoints, con un cliente REST, que permita comprobar el funcionamiento sin frontend.

Algunas pruebas de ejemplo:

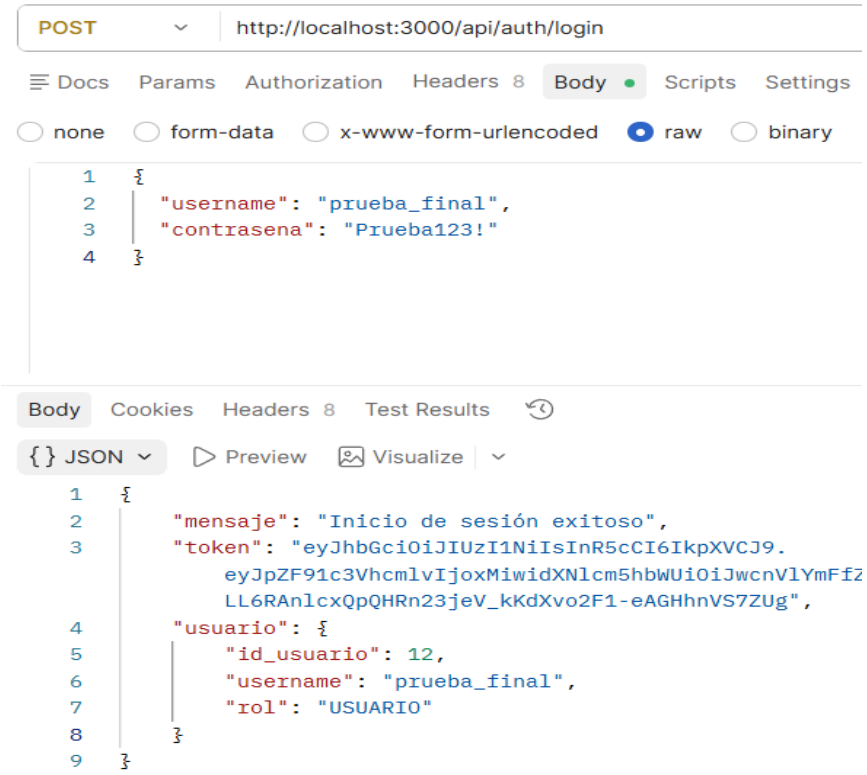
- Prueba: Comprobar que la API funciona. Método GET.



- Prueba: Registrar Usuario. Método POST. Endpoint api/auth/registro.
  Resultado obtenido 201 Created. El usuario se registro correctamente y la respuesta devolvió su identificador, username, email, rol, fecha de registro y estado, sin exponer su contraseña.

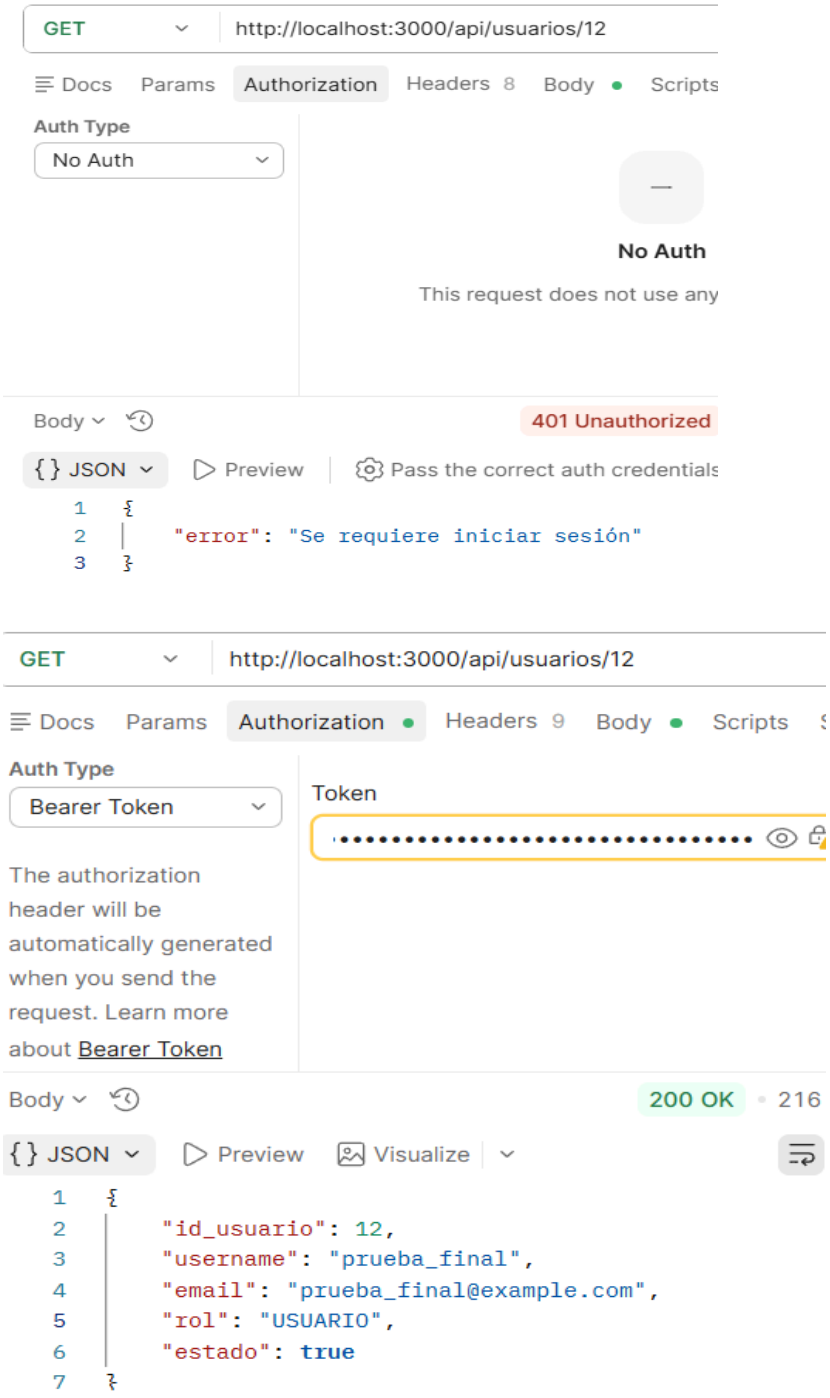


- Prueba: Probar inicio de sesión. Método POST. Endpoint api/auth/login. Resultado obtenido 200 OK cuando las credenciales son correctas, mostrando un mensaje de inicio de sesión exitoso, un token de autenticación (JWT) y los datos básicos del usuario: ID, nombre de usuario y rol. Sin mostrar contraseña.

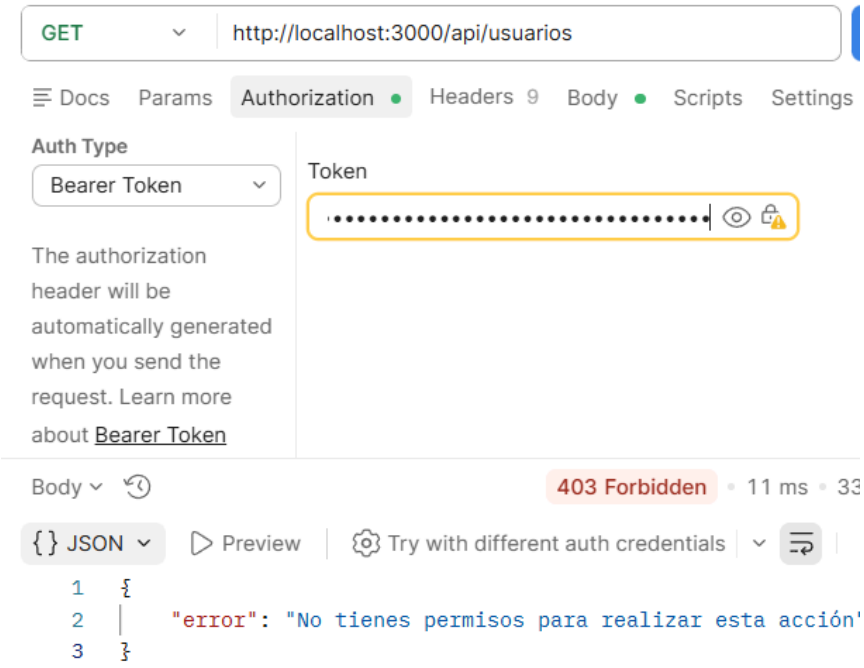


- Prueba: Acceso sin token de autenticación. Se realizó una petición GET al endpoint /api/usuarios/12 sin enviar un token de autenticación. Se esperaba que la API rechazara la solicitud con un código 401 Unauthorized, ya que el usuario no está

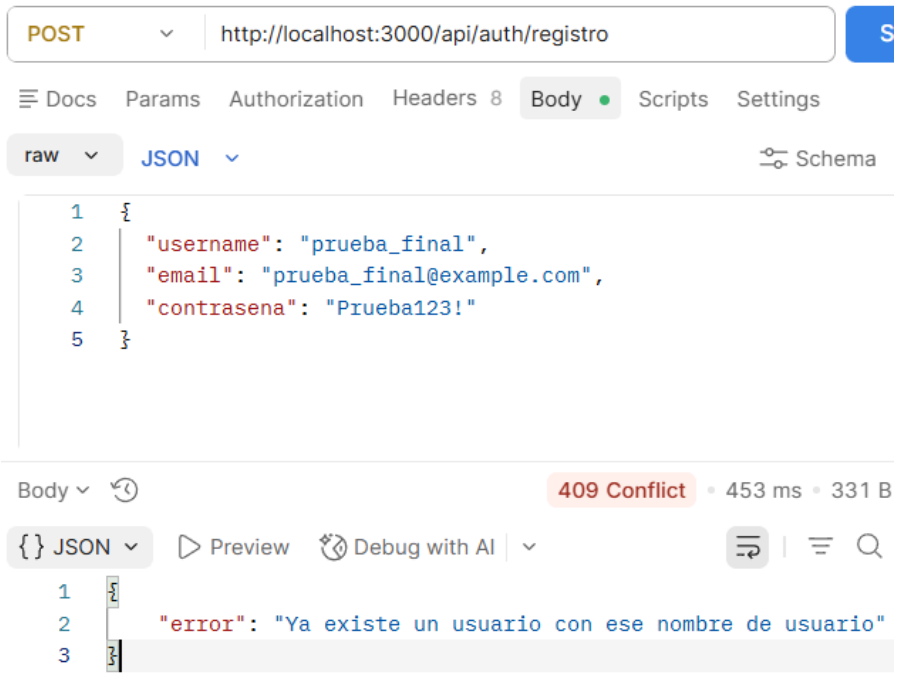
autenticado. Esta prueba permite comprobar que la ruta está protegida y que no se puede acceder a los datos sin un token válido.



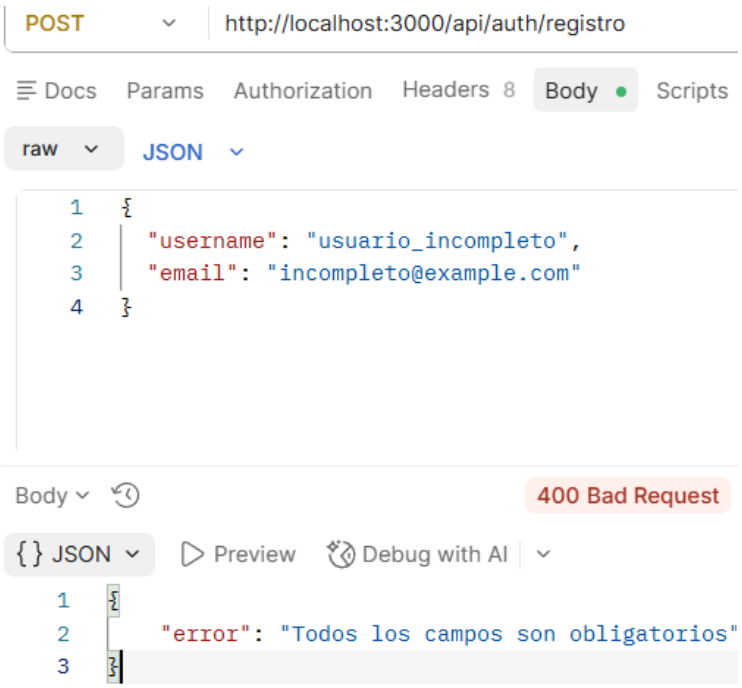
Prueba: Usuarios sin permisos. Método GET. Endpoint api/usuarios. La API debe responder 403 Forbidden, ya que el usuario autenticado tiene el rol USUARIO y no cuenta con permisos de administrador.



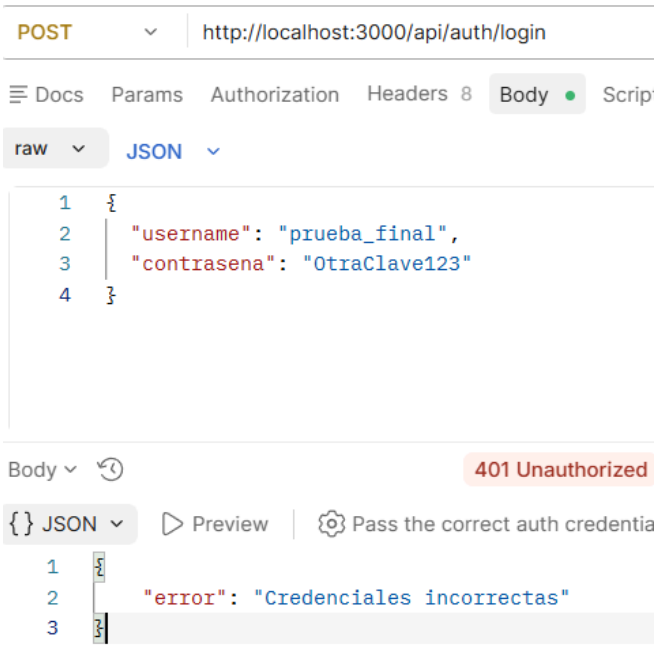
- Prueba: Registro de usuario duplicado. Método POST. Endpoint api/auth/registro. Datos enviados: Se intentó registrar un usuario con un nombre de usuario que ya existe. Resultado esperado: La API debe rechazar el registro y devolver 409 Conflict. Resultado obtenido: Se recibió el código 409 Conflict con el mensaje: "Ya existe un usuario con ese nombre de usuario".



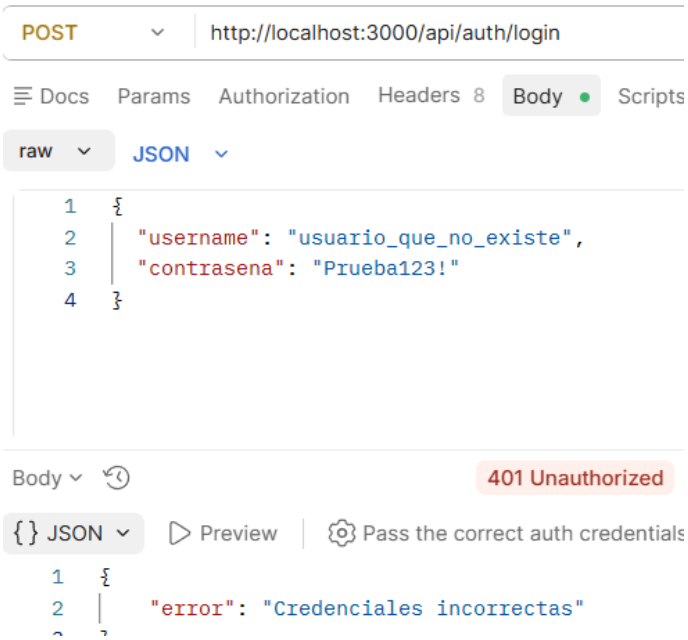
- Prueba: Registro con datos incompletos. Método POST. Endpoint: api/auth/registro. Se envió username y email, sin incluir la contraseña. Resultado esperado: La API debe rechazar la solicitud con 400 Bad Request, porque faltan los datos obligatorios.



- Prueba:Inicio de sesión con contraseña incorrecta. Método POST. Endpoint api/auth/login. Se ingresó un nombre de usuario existente con una contraseña incorrecta. Resultado esperado: La API debe rechazar el inicio de sesión con 401 Unauthorized.

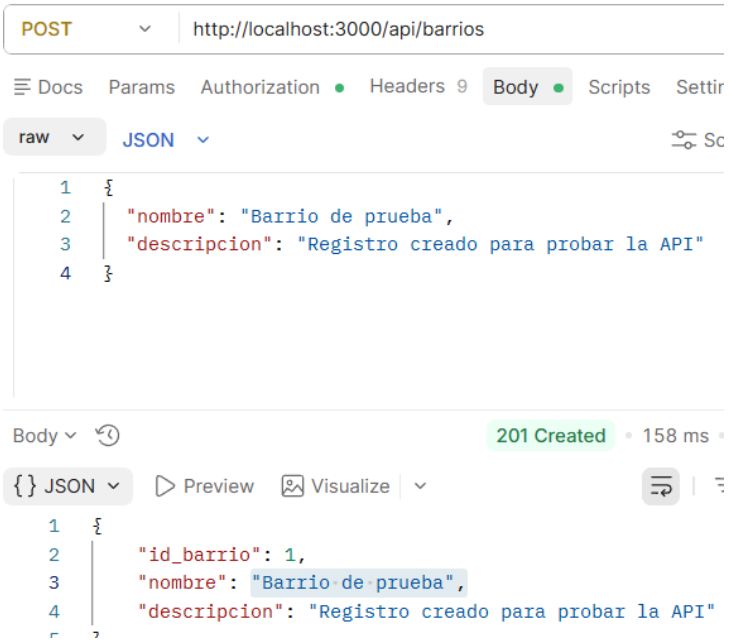


- Prueba: Inicio de sesión con usuario inexistente. Método POST. Endpoint api/auth/login. Se ingresó un nombre de usuario que no está registrado y una contraseña. Resultado esperado: La API debe rechazar el inicio de sesión con 401 Unauthorized.

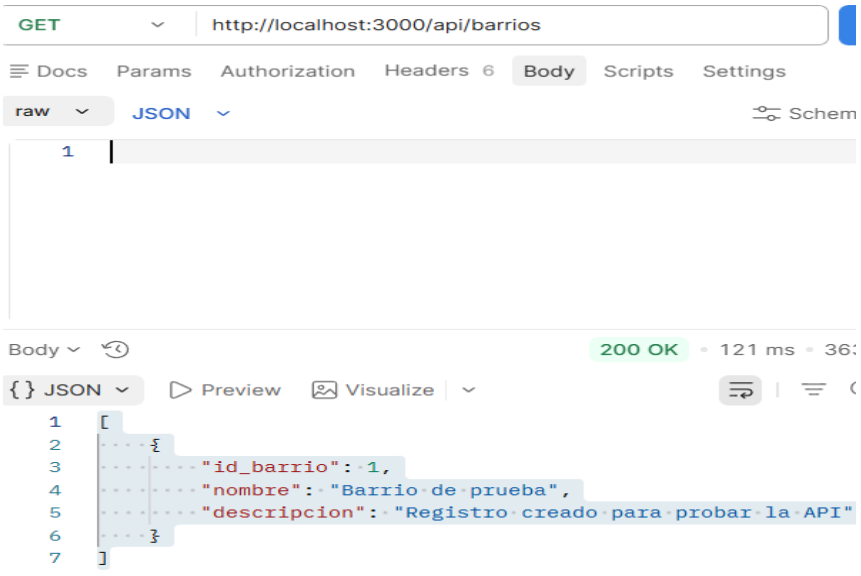


- Operaciones CRUD:
- Barrios:
- Prueba: Consultar barrios. Método: GET. Endpoint /api/barrios.Resultado esperado: La API debe devolver una lista de barrios registrados. Si no existen registros, debe devolver una lista vacía. Resultado obtenido: Se recibió 200 OK con una lista vacía ([]).

- Prueba: Crear barrio. Método: POST. Endpoint/api/barrios. Autorización: Bearer Token (usuario administrador). Resultado esperado: Crear el barrio y devolver sus datos con un identificador. Resultado obtenido: Se creó el barrio con id_barrio:1, nombre y descripción.



- Prueba: Listar barrios. Método: GET. Endpoint/api/barrios.Resultado esperado: Obtener la lista de barrios registrados. Resultado obtenido: Código 200 OK. Se devuelve una lista con el barrio Barrio de prueba, su identificador (id_barrio: 1) y su descripción.



- Prueba: Modificar un barrio. Método: PUT. Endpoint/api/barrios/1. Autorización: Bearer Token (administrador). Resultado esperado: Modificar los datos del barrio seleccionado. Resultado obtenido: Código 200 OK. El barrio con ID 1 devuelve el nombre y la descripción actualizados.

  Body enviado:

```
{
  "nombre": "Barrio actualizado",
  "descripcion": "Barrio modificado para probar la API"
}
```

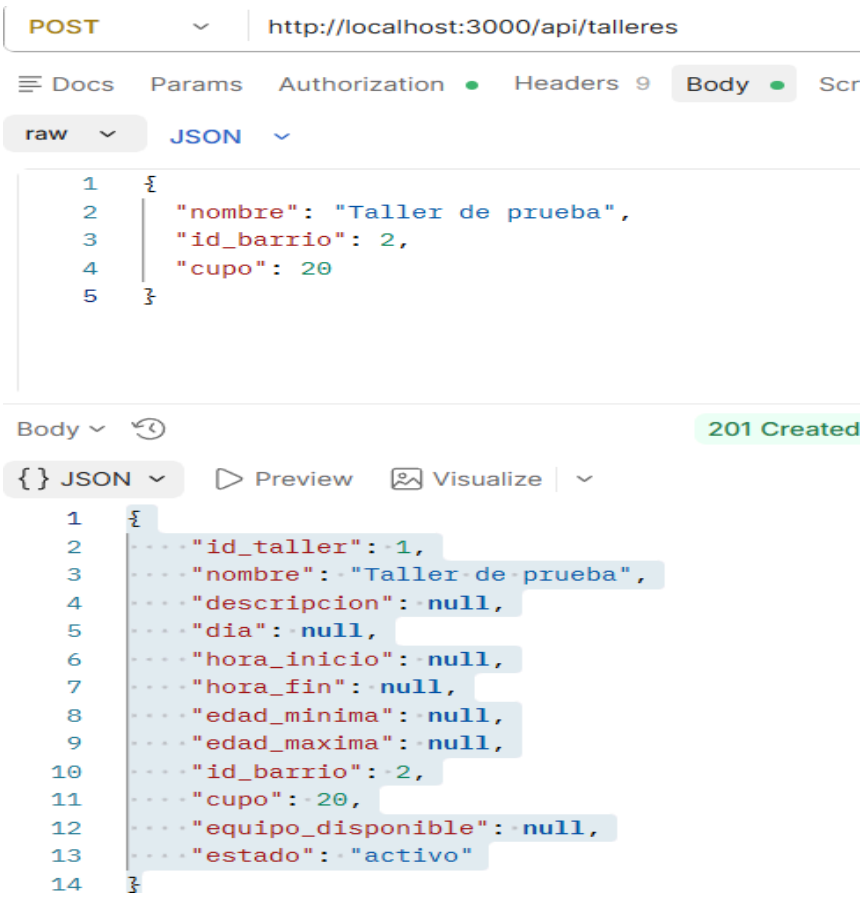

Respuesta:


```
{
    "id_barrio": 1,
    "nombre": "Barrio actualizado",
    "descripcion": "Barrio modificado para probar la API"
}
```


- Prueba: Eliminar un barrio. Método: DELETE. Endpoint /api/barrios/1. Autorización: Bearer Token (administrador). Resultado esperado: Eliminar el barrio seleccionado. Resultado obtenido: Se devuelve el mensaje "Barrio eliminado correctamente".

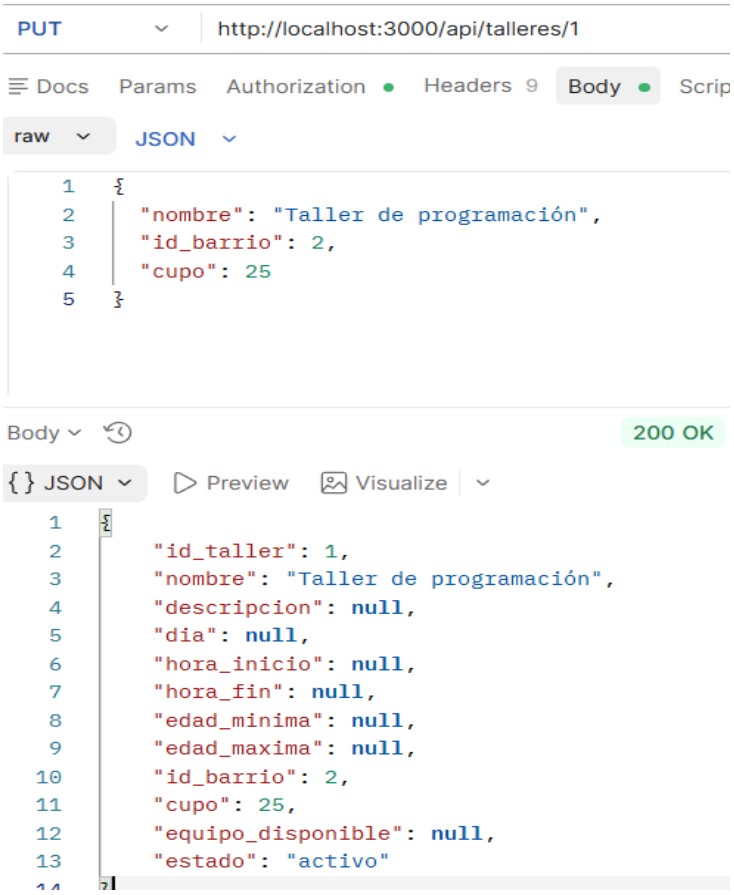
- Talleres:
- Prueba: Consultar talleres. Método: GET. Endpoint /api/talleres.Resultado esperado: Obtener la lista de talleres registrados. Resultado obtenido: Código 200 OK. La API devuelve una lista vacía ([]), porque no hay talleres registrados.

- Prueba: Crear un taller. Método: POST. Endpoint/api/talleres.Autorización: Bearer Token (administrador). Resultado obtenido: Se creó el taller con id_taller: 1, nombre "Taller de prueba", id_barrio: 2, cupo de 20 y estado "activo". Los campos opcionales no enviados aparecen como null.



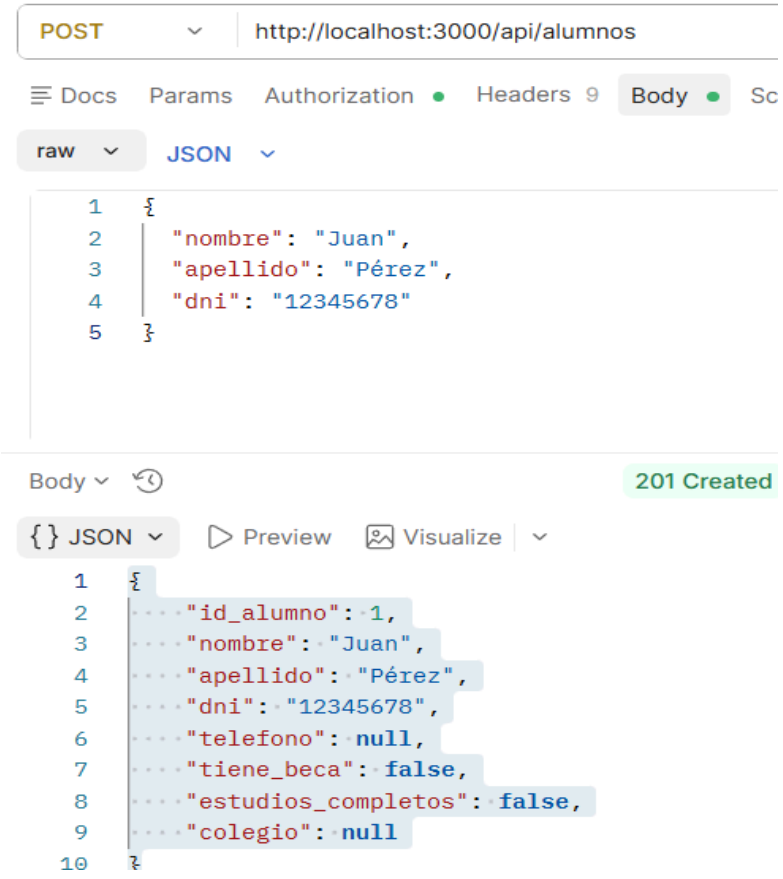
- Prueba: Listar talleres. Método: GET. Endpoint/api/talleres.Resultado esperado: Obtener la lista de talleres registrados. Resultado obtenido: Código 200 OK. La API devuelve el taller con ID 1, nombre "Taller de prueba", barrio asociado 2, cupo de 20 y estado "activo".

- Prueba: Modificar un taller. Método: PUT. Endpoint /api/talleres/1 Autorización: Bearer Token (administrador). Resultado esperado: Actualizar los datos del taller seleccionado. Resultado obtenido: El taller con ID 1 devuelve el nombre actualizado, el cupo de 25, el barrio asociado 2 y el estado "activo".

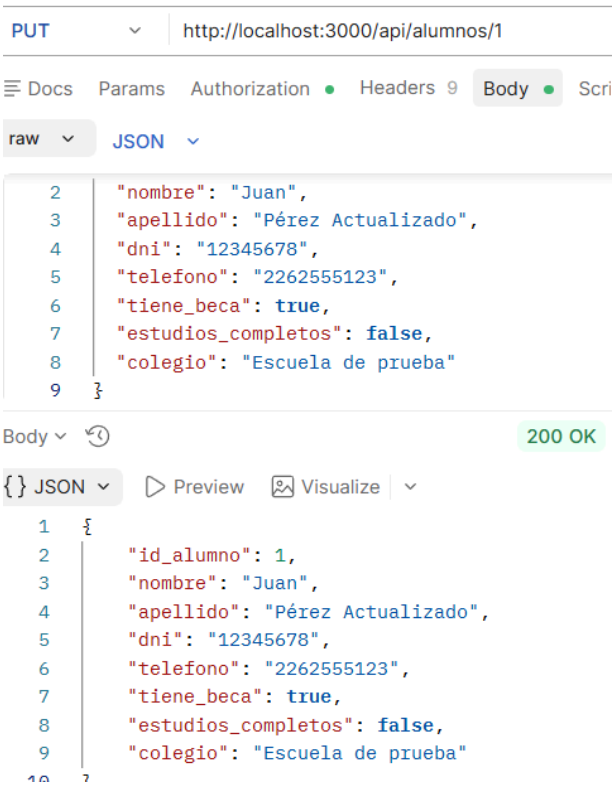


- Prueba: Eliminar un taller. Método: DELETE. Endpoint /api/talleres/1. Autorización: Bearer Token (administrador). Resultado esperado: Eliminar el taller seleccionado. Resultado obtenido: Código 200 OK. La API devuelve el mensaje "Taller eliminado correctamente".

- Alumnos:
- Prueba: Listar alumnos. Método: GET. Endpoint /api/alumnos.Autorización: Bearer Token (administrador). Resultado esperado: Obtener la lista de alumnos registrados.
- Prueba: Crear un alumno. Método: POST. Endpoint api/alumnos. Autorización: Bearer Token (administrador). Resultado esperado**:** Crear un alumno con los campos obligatorios y devolver sus datos.

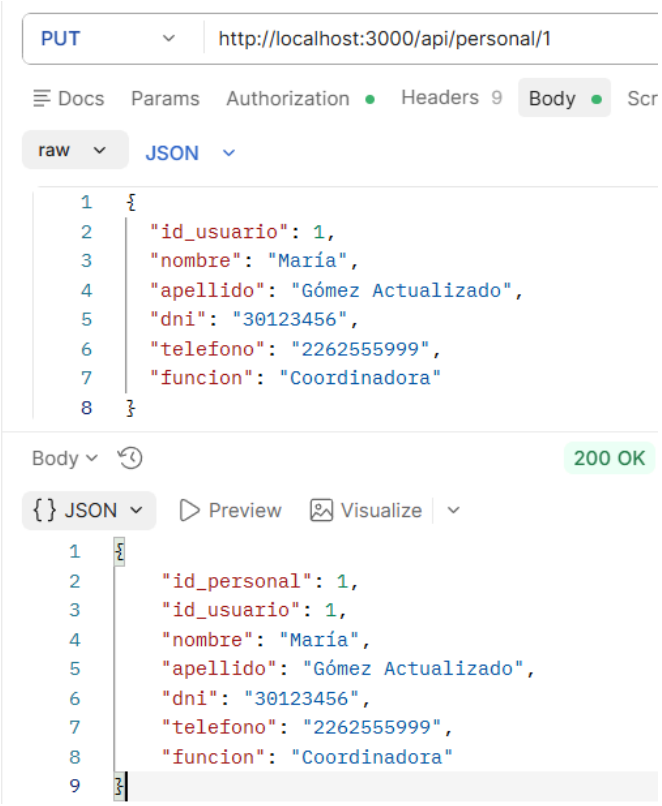


- Prueba: Listar alumnos. Método: GET. Endpoint api/alumnos. Autorización: Bearer Token (administrador). Resultado esperado: Obtener la lista de alumnos registrados. Resultado obtenido: Código 200 OK. La consulta se ejecutó correctamente y devuelve los alumnos registrados.
- Prueba: Obtener un alumno por ID. Método: GET. Endpoint /api/alumnos/1 Autorización: Bearer Token (administrador). Resultado esperado: Obtener los datos del alumno cuyo ID es 1. Resultado obtenido: Código 200 OK. La API devuelve correctamente los datos de Juan Pérez, con DNI 12345678 y los valores correspondientes a los campos opcionales.
- Prueba: Modificar un alumno. Método: PUT. Endpoint /api/alumnos/1 Autorización: Bearer Token (administrador). Resultado esperado: Modificar los datos del alumno seleccionado. Resultado obtenido: Se actualizó el alumno con ID 1. El apellido, teléfono, beca y colegio coinciden con los nuevos valores enviados.



- Prueba: Eliminar un alumno. Método: DELETE. Endpoint /api/alumnos/1 Autorización: Bearer Token (administrador). Resultado esperado: Eliminar al alumno seleccionado. Resultado obtenido: La API devuelve el mensaje "Alumno eliminado correctamente".

- Personal:
- Prueba: Listar personal. Método: GET. Endpoint /api/personal Autenticación: Bearer Token (administrador). Resultado esperado: Listar el personal registrado. Resultado obtenido: Lista vacía []. Estado HTTP: 200 OK.
- Prueba: Crear personal. Método POST. Endpoint /api/personal. Autenticación Bearer Token (administrador). Resultado esperado: Crear un registro de personal. Resultado obtenido: Personal creado con id_personal: 1.
- Prueba: Listar personal. Método GET. Endpoint /api/personal. Resultado esperado: Obtener el listado del personal. Resultado obtenido: Devuelve el registro con id_personal: 1. Estado HTTP 200 OK.
- Prueba: Consultar por id. Método POST. Endpoint /api/personal/1. Resultado esperado: Obtener los datos del personal con ID 1. Resultado obtenido: Devuelve correctamente los datos. Estado HTTP: 200 OK.
- Prueba: Actualizar personal. Método PUT. Endpoint /api/personal/1. Resultado esperado: Modificar los datos del personal con ID 1. Resultado obtenido: Datos actualizados correctamente. Estado HTTP 200 OK.



- Prueba: Eliminar Personal. Método: DELETE . Endpoint /api/personal/1. Resultado esperado: Eliminar el registro de personal con ID 1. Resultado obtenido: "Personal eliminado correctamente". Estado HTTP 200 OK.

Códigos de estado utilizados

- **200** OK: la operación se realizó y devuelve datos.
- **201** Creado: se creó un recurso nuevo.
- **204** Sin contenido: la operación se realizó y no hay nada para devolver.
- **400** Solicitud inválida: faltan datos o tienen un formato incorrecto.
- **401** No autenticado: no inició sesión o el token es inválido o venció.
- **403** Prohibido: inició sesión, pero su rol no tiene permiso.
- **404** No encontrado: el recurso no existe.
- **409** Conflicto: choca con el estado actual (dato duplicado, sin cupo, solicitud ya resuelta, registro con dependencias).
- **500** Error interno del servidor.